First Two-Class Appointment Simulation: Recommended Formulation

Alexandre SEPULVEDA de DIETRICH
Under the supervision of Pr. Carri Chan

April 8, 2026

1 Purpose

This note defines the first discrete-event appointment simulation used in this project and clarifies the notation so it matches the underlying dynamics of the problem. The main design goal is to keep the first simulator simple enough to run, inspect, and extend, while using a formulation that remains valid once we add richer allocation policies, heterogeneous patient behavior, or optimization layers.

The note aligns the simulator with the queueing models in:

- Green and Savin (2008), *Reducing Delays for Medical Appointments: A Queueing Approach*,
- Liu and Ziya (2014), *Panel Size and Overbooking Decisions for Appointment-based Services under Patient No-shows*,
- Liu (2015), *Optimal Choice for Appointment Scheduling Window under Patient No-Show Behavior*.

All three papers study the same structural mechanism:

more backlog $\Rightarrow$ more delay $\Rightarrow$ worse attendance $\Rightarrow$ lower effective capacity.

The simulation keeps that mechanism, but implements it on an explicit rolling calendar with two patient classes, explicit cancellations, and a pluggable slot-allocation rule.

2 Why the Naive Formulation is Not Sufficient

An initial formulation might write

$$X_{i,\tau}^{D} = \text{number of class-}i\text{ patients booked for day } D+\tau$$

and then also use τ inside $b_i(\tau)$ and $\xi_i(\tau)$. That notation is not the best formulation for this problem.

2.1 Problem 1: It Overloads the Delay Variable

There are two different delays in the system:

1. the *remaining delay* from the current day to the appointment,
2. the *original delay seen at booking time*.

These play different roles. The remaining delay is a state variable. The original delay at booking is the natural argument of the behavioral functions.

If the same symbol τ is used for both, then the model becomes ambiguous. In particular, it can incorrectly suggest that no-show probability should be recomputed from remaining delay as the appointment approaches. That is not the intended behavioral mechanism here.

2.2 Problem 2: The Aggregate Day-Level Count is Not the True State

The vector $\{X_{i,\tau}^{D}\}$ is a useful summary, but it is not the full simulator state once we include:

- slot-level capacity,
- day-by-day cancellations that reopen specific future slots,
- alternative booking policies that may choose different open slots within the same day.

For those features, the actual state must retain the slot-level calendar.

2.3 Problem 3: It Hides the Link to the Literature

In the source papers, attendance depends on the backlog or delay the patient sees when the appointment is accepted. The relevant quantity is therefore the *booking-time delay*, not the residual delay just before service.

The simulator should preserve that same logic in explicit-calendar form.

3 Recommended Formulation

3.1 Decision Epochs

Let time advance in slot-sized decision epochs. Write

$$t = (D, s),$$

where:

- D is the calendar day,
- $s \in \{0, 1, \ldots, S-1\}$ is the active slot within day D.

In the first version of the simulator,

$$H = 15 \text{ days}, \qquad S = 25 \text{ slots per day},$$

where H is the booking horizon.

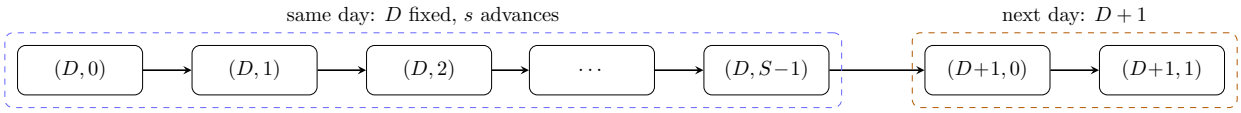
3.2 How Time Advances

The ordered pair $t = (D, s)$ advances lexicographically:

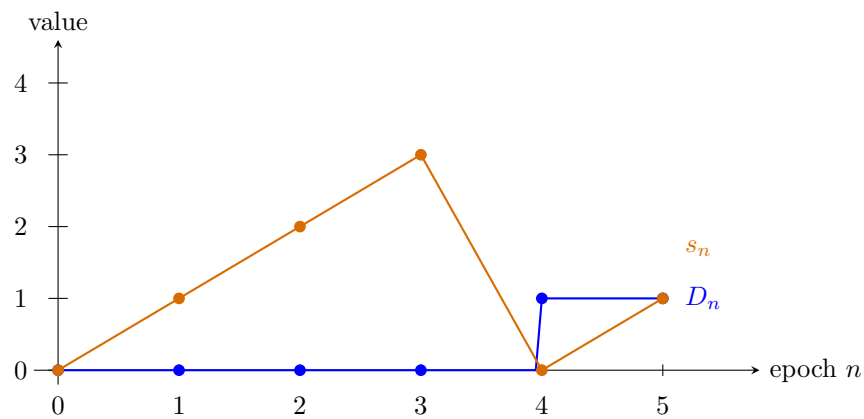
$$(D, s)^{+} = \begin{cases} (D, s+1), & \text{if } s < S-1, \\ (D+1, 0), & \text{if } s = S-1. \end{cases}$$

So the slot index s moves within a day, and the day index D increases only when the current day has exhausted all S slots.

The same evolution can be visualized as a state-transition graph:



To see both coordinates on the same graph, it is useful to plot them against the epoch index. The following illustration starts at day 0 and uses $S = 4$ only so the rollover is visible on a small plot:



On this graph, D_n is a step process while s_n increases within a day and resets to 0 when the process rolls to the next day.

This is the time scale on which arrivals, booking decisions, cancellations applied at the start of each day, and service realizations are processed.

3.3 True State: Slot-Level Calendar

The true simulator state at time $t = (D, s)$ is the rolling slot-level calendar. The mathematical state is indexed by residual day r and slot index m. For visualization, however, it is often more natural to display the calendar with days as *columns* and slots as *rows*. In that display, the column index corresponds to the day ahead relative to the current day D: $r = 0$ means "today", $r = 1$ means "tomorrow", and so on. The row index corresponds to the slot within a day: $m = 0$ is the first slot, $m = 1$ the second, and so on until $m = S - 1$.

Formally, for each residual day offset

$$r \in \{0, 1, \ldots, H - 1\}$$

and each slot index

$$m \in \{0, 1, \ldots, S - 1\},$$

define

$$Y_t(r, m).$$

So $Y_t(r, m)$ is the content of the cell indexed by day-ahead r and slot m in the rolling calendar. If

$$Y_t(r, m) = 0,$$

then that slot is open. If

$$Y_t(r, m) = (i, \tau)\,,$$

then that slot is occupied by a class-i patient who originally accepted delay τ.

The class index i links that booked patient to the class object, written later as

$$C_i = \{\lambda_i, b_i(\tau), \phi_i(\tau), \xi_i(\tau)\}.$$

So the cell itself stores only the patient features that must remain attached to the appointment: the class identity i and the original offered delay τ. The remaining class characteristics are recovered from C_i when the simulator needs them:

- λ_i governs future arrivals from class i,
- $b_i(\tau)$ governs booking acceptance when a class-i request is offered delay τ,
- $\phi_i(\tau)$ governs daily pre-appointment cancellation for class-i patients booked with delay τ,
- $\xi_i(\tau)$ governs no-show at appointment time.

Schematically, the rolling calendar has the following structure:

Slot in day	**day** D $(r=0)$	**day** $D+1$ $(r=1)$	$\cdots$	**day** $D+H-1$ $(r=H-1)$
slot 0 $(m=0)$	$Y_t(0,0)$	$Y_t(1,0)$	$\cdots$	$Y_t(H-1,0)$
slot 1 $(m=1)$	$Y_t(0,1)$	$Y_t(1,1)$	$\cdots$	$Y_t(H-1,1)$
later slots	$\vdots$	$\vdots$	$\ddots$	$\vdots$
slot $S-1$ $(m=S-1)$	$Y_t(0,S-1)$	$Y_t(1,S-1)$	$\cdots$	$Y_t(H-1,S-1)$

So each column is a day in the rolling horizon, and each row is a slot inside that day. The state variable $Y_t(r,m)$ is just the content of one cell of this table.

To make the notation concrete, suppose class 1 is an MRI-like diagnostic request and class 2 is a behavioral-health follow-up request. Suppose also that each day has four appointment times: 08:00, 10:00, 13:00, and 15:00.

A realistic miniature appointment book with $H=3$ and $S=4$ looks like this:

Slot in day	**day** D	**day** $D+1$	**day** $D+2$
08:00 $(m=0)$	open	$(1,0)$	open
10:00 $(m=1)$	$(1,0)$	open	$(2,2)$
13:00 $(m=2)$	open	$(2,1)$	$(1,2)$
15:00 $(m=3)$	$(2,0)$	open	open

This example is read column by column. The first column is today's remaining schedule, the second column is tomorrow's schedule, and the third column is the schedule for the day after tomorrow. For instance, the entry in day-column $r = 1$ and slot-row $m = 2$ is $(2, 1)$, meaning that the 13:00 slot of day $D + 1$ is occupied by a class-2 behavioral-health patient who accepted a one-day delay. Because the patient belongs to class 2, the simulator knows to use the class-2 behavior object $C_2 = \{\lambda_2, b_2(\tau), \phi_2(\tau), \xi_2(\tau)\}$. That means:

- future class-2 arrivals are generated with rate λ_2,
- the patient's daily pre-appointment cancellation behavior is governed by $\phi_2(1)$,
- if the patient reaches service, the no-show probability is $\xi_2(1)$ because the original booked delay is $\tau = 1$.

This is the minimal state representation that supports:

- explicit holes created by daily cancellation,
- slot-level FCFS or non-FCFS allocation,
- class-specific booked-delay tracking,
- later policy experiments that depend on slot position.

3.4 Derived Day-Start Summary

Although the slot-level calendar is the true state, it is still useful to retain an aggregate day-start summary that resembles the literature. At the start of day D, after the previous day's end-of-day cancellations and before processing slot 0, define

$$X_{i,r}^D = \sum_{m=0}^{S-1} \mathbf{1}\{Y_{(D,0)}(r, m) \text{ is occupied by a class-}i \text{ patient}\},$$

for

$$i \in \{1, 2\}, \qquad r \in \{0, 1, \ldots, H - 1\}.$$

In words, $X_{i,r}^D$ counts how many class-i patients appear in row r of the calendar at the start of day D. Equivalently,

$$X_{i,r}^D = \text{number of class-}i \text{ patients scheduled at the start of day } D \text{ for day } D + r.$$

This is the correct interpretation of the state-count variable. The index r is a *residual delay*, not a booking-time delay.

3.5 Worked Example of Y and X

It is useful to see one concrete calendar snapshot and translate it directly into the state and equations. Suppose:

$$D = 0, \qquad s = 0, \qquad H = 3, \qquad S = 4.$$

So we are at the start of the current operating day, just before the 08:00 slot, and the rolling calendar covers today, tomorrow, and the day after tomorrow.

Assume the slot-level state is:

	08:00	10:00	13:00	15:00
$r = 0$ (today)	0	$(1, 0)$	$(2, 0)$	0
$r = 1$ (tomorrow)	$(1, 1)$	0	$(1, 1)$	$(2, 1)$
$r = 2$ (day after)	0	$(2, 2)$	$(1, 2)$	0

Interpretation:

- row $r = 0$ corresponds to today's remaining slots,
- row $r = 1$ corresponds to tomorrow,
- row $r = 2$ corresponds to the day after tomorrow,
- entry 0 means the slot is open,
- entry (i, τ) means the slot is occupied by a class-i patient who accepted an offered delay τ.

For example:

- $Y_{(0,0)}(0, 1) = (1, 0)$ means that today's 10:00 slot is booked by a class-1 MRI patient who accepted a same-day offer,
- $Y_{(0,0)}(1, 0) = (1, 1)$ means that tomorrow's 08:00 slot is booked by a class-1 MRI patient who accepted a one-day delay,
- $Y_{(0,0)}(2, 1) = (2, 2)$ means that the 10:00 slot two days from now is booked by a class-2 behavioral-health patient who accepted a two-day delay.

For the second patient, the state stores only $(2, 2)$. That is enough because class index 2 points to

$$C_2 = \{\lambda_2, b_2(\tau), \phi_2(\tau), \xi_2(\tau)\}.$$

So while the patient waits, the simulator uses the class-2 cancellation function $\phi_2(\tau)$ to apply the daily cancellation probability $\phi_2(2)$, and if the patient reaches service it uses $\xi_2(2)$ for attendance.

From this calendar, the derived day-start counts are:

$$X^0_{1,0} = 1, \quad X^0_{2,0} = 1,$$

$$X^0_{1,1} = 2, \quad X^0_{2,1} = 1,$$

$$X^0_{1,2} = 1, \quad X^0_{2,2} = 1.$$

So the vector $X^0_{i,r}$ is just a class-by-class count of the occupied entries in each row. It does *not* remember which slot inside the day is occupied, nor does it retain the full slot-level identity of individual patients. That is why $X^D_{i,r}$ is a useful summary, but not the full state needed by the simulator.

The distinction between r and τ is also visible in this example. Consider the patient stored as

$$Y_{(0,0)}(2, 2) = (1, 2).$$

On day 0, that patient contributes to $X^0_{1,2}$. If the patient remains scheduled, then at the start of day 1 the same patient will contribute to $X^1_{1,1}$, because the *residual delay* has fallen from 2 to 1. However, the patient's booking-time delay remains $\tau = 2$, and that is the quantity that should continue to drive the behavioral functions $b_1(\tau)$ and $\xi_1(\tau)$.

3.6 From Calendar Story to Equations

The realistic booking story above transfers directly into the mathematical model. Suppose a new class-2 patient arrives just before today's 08:00 slot and the FCFS rule offers the open slot at 08:00 two days from now. Then:

- the offered delay is $\tau = 2$ because the offered day is $D + 2$,
- the patient books with probability $1 - b_2(2)$,
- if the patient accepts, the new slot content becomes

$$Y_{(0,0)}(2, 0) = (2, 2),$$

- at the end of each pre-appointment day, that patient cancels with daily probability $\phi_2(2)$,

- if the patient reaches the appointment, attendance occurs with probability $1 - \xi_2(2)$.

So the same appointment can be read in three equivalent ways:

- as a realistic story: "a behavioral-health follow-up booked for two days from now,"
- as a state entry: $Y_t(r, m) = (2, 2)$ in the appropriate cell,
- as behavioral equations: booking through $b_2(2)$, cancellation through $\phi_2(2)$, and attendance through $\xi_2(2)$.

3.7 Booking-Time Delay

Now define the behavioral delay variable separately:

$$\tau = \text{delay in days between the booking day and the appointment day offered to the patient.}$$

If a patient arrives on day d and is offered a slot on day $d + \tau$, then:

- $b_i(\tau)$ is the probability that class-i declines the offer,
- if the patient accepts, τ is stored with the booking,
- $\xi_i(\tau)$ is the no-show probability used at appointment time if the patient remains scheduled.

This separation is the key modeling choice. Residual delay belongs to the state. Booking delay belongs to the behavioral rules.

4 Patient Classes and Arrival Mix

The class-specific behavior of each class $i \in \{1, 2\}$ is described by

$$B_i = \{b_i(\tau), \phi_i(\tau), \xi_i(\tau)\}.$$

In the current two-class prototype, arrivals are parameterized at the system level by

$$\lambda = \text{total expected arrival rate per slot}, \qquad p \in [0, 1] = \text{share of class-1 demand.}$$

The class-specific arrival rates are then induced by

$$\lambda_1 = p\lambda, \qquad \lambda_2 = (1 - p)\lambda.$$

Equivalently, the full class objects can still be written as

$$C_i = \{\lambda_i, b_i(\tau), \phi_i(\tau), \xi_i(\tau)\},$$

but in the first simulation experiments the primitive demand parameters are (λ, p) rather than (λ_1, λ_2).

The interpretation is therefore:

- λ: total expected arrival rate per slot,
- p: fraction of arrivals belonging to class 1,
- λ_i: implied arrival rate of class-i requests per slot,
- $b_i(\tau)$: probability the patient declines the offered slot because the offered delay is too large,
- $\phi_i(\tau)$: daily probability that a class-i patient booked with original delay τ cancels at the end of a pre-appointment day,
- $\xi_i(\tau)$: no-show probability at appointment time, conditional on the patient still being scheduled.

In the current first version, the daily cancellation function $\phi_i(\tau)$ is induced by a class-level eventual cancellation parameter $\bar{\phi}_i$ according to

$$\phi_i(\tau) = \begin{cases} 0, & \tau = 0, \\ 1 - (1 - \bar{\phi}_i)^{1/\tau}, & \tau \geq 1, \end{cases}$$

so that a patient who booked with delay τ and faces one cancellation trial at the end of each pre-appointment day still has total cancellation probability $\bar{\phi}_i$ before the appointment. Later versions may generalize this further to a directly delay-sensitive cancellation object.

5 Event Dynamics

5.1 Slot-Level Event Order

At each active slot (D, s), the simulator executes the following within-day steps.

1. Generate arrivals
$$A_i(D, s) \sim \text{Poisson}(\lambda_i), \qquad i \in \{1, 2\}.$$
In the current parameterization, $\lambda_1 = p\lambda$ and $\lambda_2 = (1 - p)\lambda$.

2. Randomize the order of individual arrivals within the slot.

3. For each arrival, call the allocation policy to select an open future slot.

4. If a slot on day $D + r$ is offered, interpret
$$\tau = r.$$

5. The patient accepts with probability
$$1 - b_i(\tau).$$

6. When the active slot is served, the patient in that slot, if any, attends with probability
$$1 - \xi_i(\tau),$$
where τ is the original delay stored when the patient booked.

After all S slots of day D have been processed, every still-scheduled future appointment stored as (i, τ) cancels independently with probability $\phi_i(\tau)$. Any canceled future slot is therefore reopened before day $D + 1$ begins. This timing is operationally natural:

- a booking with $\tau = 1$ gets exactly one chance to cancel, namely at the end of the booking day,
- a booking with $\tau = 2$ gets two daily cancellation opportunities before service,
- the total cancellation probability across those daily trials remains $\bar{\phi}_i$.

5.2 FCFS Baseline Policy

In the baseline version, the allocation rule is:

> choose the earliest open slot strictly after the active slot.

That rule is first-come first-served in calendar order, but the simulator is intentionally written so the slot-selection rule can later be replaced without changing the event loop. The current codebase now includes two additional initial-model examples:

- a strict reserved-capacity policy that protects a specified fraction or count of daily slots for each class,
- a class-specific booking-window policy that restricts class i to delays no larger than k_i.

6 Performance Measures

6.1 Arrival-Cohort Measures

For patients who arrive during the measured window, define:

$$\begin{aligned}
\text{Arrivals}_i &= \text{number of class-}i\text{ appointment requests,}\\
\text{Booked}_i &= \text{number of class-}i\text{ requests that accept an offer,}\\
\text{Balked}_i &= \text{number of class-}i\text{ requests that were offered a slot and declined it,}\\
\text{NoOffer}_i &= \text{number of class-}i\text{ requests for which no eligible slot was available,}\\
\text{NotBooked}_i &= \text{Balked}_i + \text{NoOffer}_i,\\
\text{Canceled}_i &= \text{number of class-}i\text{ booked patients who pre-cancel,}\\
\text{NoShow}_i &= \text{number of class-}i\text{ booked patients who remain scheduled but do not attend,}\\
\text{Served}_i &= \text{number of class-}i\text{ booked patients who attend,}\\
\bar{\tau}_i^{\text{book}} &= \text{mean original booking delay among class-}i\text{ booked patients.}
\end{aligned}$$

The main service metric in the first version is

$$\text{ServiceRate}_i = \frac{\text{Served}_i}{\text{Booked}_i}.$$

Aggregate values are formed by summing over classes and taking the corresponding weighted mean.

6.2 Slot-Based Measures

Over measured service slots, define:

$$\begin{aligned}
\text{BookedSlots} &= \text{slots with a booked patient at service time,}\\
\text{ServedSlots} &= \text{slots where the booked patient attends,}\\
\text{NoShowSlots} &= \text{slots where a booked patient is present on the schedule but does not attend,}\\
\text{EmptySlots} &= \text{slots with nobody scheduled at service time.}
\end{aligned}$$

Two utilization measures are reported:

$$\rho^{\text{booked}} = \frac{\text{BookedSlots}}{HS_{\text{measured}}}, \qquad \rho^{\text{attended}} = \frac{\text{ServedSlots}}{HS_{\text{measured}}}.$$

The first measures scheduled load. The second measures realized attended load.

7 Comparison to the Literature

7.1 Common Structural Mapping

The source papers use more stylized queue states than the simulator, but the economic and behavioral interpretation is the same. In each paper, attendance depends on the congestion level a patient sees when entering the system.

In the analytical papers, that congestion variable is usually:

- queue length,
- backlog level,
- or the number of appointments already ahead in the system.

In the simulator, the explicit-calendar analog is the offered delay τ. That is why τ, rather than residual delay r, should drive $b_i(\cdot)$ and $\xi_i(\cdot)$.

7.2 Green and Savin (2008)

Green and Savin study a single-server appointment queue in which no-show probability increases with the backlog seen when the appointment is made. Their no-show curve is of the exponential-saturation form

$$\gamma(k) = \gamma_{\max} - (\gamma_{\max} - \gamma_0)e^{-k/C},$$

where k is a backlog measure.

The mapping to the simulator is:

- their backlog state corresponds to our explicit slot calendar,
- their delay-sensitive no-show mechanism corresponds to our $\xi_i(\tau)$,
- their stylized queue-length state is refined into the explicit schedule $Y_t(r, m)$.

The simulation differs from Green and Savin in several ways:

- it allows two patient classes rather than one homogeneous stream,
- it includes explicit balking before booking,
- it includes daily pre-appointment cancellation that creates explicit holes in the calendar,
- it does not yet model no-show rescheduling.

The most important conceptual similarity is this: attendance deteriorates with the delay the patient faced when entering the schedule.

7.3 Liu and Ziya (2014)

Liu and Ziya study panel size and overbooking decisions in stylized queueing models with delay-dependent attendance. Their state variable is again a queue-length-type object, and a patient's show-up probability depends on the congestion level observed at booking time.

Relative to Liu and Ziya, the simulator:

- fixes daily slot capacity at 25 in the first version,
- does not yet optimize overbooking or panel size,
- keeps the booking calendar explicit rather than compressing it into a single queue-length state,
- is designed to allow class-specific balking, cancellation, and attendance curves.

Their model is therefore more compact and optimization-oriented. Our simulator is more descriptive and state-rich. The key shared mechanism remains the same:

$$\text{booking-time congestion} \longrightarrow \text{attendance}.$$

7.4 Liu (2015)

Liu studies the optimal appointment-window size K under delay-dependent no-shows. That paper's main control is the admissible backlog or booking window.

The simulator's rolling horizon length H plays the same operational role as a hard scheduling window, but here the horizon is fixed in the baseline implementation. This makes the simulator a natural platform for later experiments in which:

- H is varied,
- different allocation policies are compared,
- or class-specific admission rules are introduced.

Again, the important mapping is not the exact state variable but the interpretation: the patient's future attendance depends on the delay accepted at booking time.

7.5 Side-by-Side Comparison

Model / paper	State abstraction	Delay variable that drives attendance	Main difference from this simulator
Green and Savin (2008)	queue length / backlog in an appointment queue	backlog seen when appointment is made	analytical single-class queue; no explicit slot calendar
Liu and Ziya (2014)	queue length in $M/M/1$ or $M/D/1$ style models	congestion at booking time	optimization over panel size and overbooking, not explicit calendar simulation
Liu (2015)	queue/window state with finite scheduling window K	delay implied by admissible backlog at booking	optimization over window size rather than fixed-horizon simulation
This simulator	explicit rolling calendar $Y_t(r, m)$ plus derived $X^D_{i,r}$	original offered delay τ	richer operational detail, two classes, daily cancellation holes, policy plug points

8 Why this Formulation is the Most Appropriate for This Problem

This formulation is the most appropriate starting point for the present project for four reasons.

8.1 1. It Matches the Behavioral Mechanism

The patient decides whether to book when seeing an offered delay. If they book, their future no-show behavior should be tied to that accepted delay. That is exactly what τ captures.

8.2 2. It Preserves the Literature's Logic While Using an Explicit Calendar

The source papers compress the calendar into queue length for tractability. The simulator does not need to do that. It can retain the slot-level calendar and still summarize the system in a literature-friendly way through $X^D_{i,r}$.

8.3 3. It Is Markov for the Planned Extensions

Once cancellations create holes and policies can choose among open slots, a pure day-level count is no longer enough. The calendar state $Y_t(r, m)$ is the right object to carry forward if the project later adds:

- priority rules,
- reserved capacity by class,
- different appointment windows by class,
- overbooking,
- dynamic acceptance or rejection decisions.

8.4 4. It Cleanly Separates State and Behavior

The formulation distinguishes:

- *where the system is now*: represented by residual delay r in the state,
- *what delay the patient accepted*: represented by τ in the behavior functions.

That separation is the cleanest way to avoid conceptual mistakes in both code and analysis.

9 Implementation Mapping

The current codebase follows this notation directly:

- the slot calendar stores each patient's class, appointment location, booking day, and the mathematical delay τ,
- the arrival log records the offered delay as `tau_offered`,
- the cohort and slot logs record the same mathematical quantity τ under the implementation field name `tau_booked`,
- the day-start state log records the residual day offset as `residual_delay`,
- the class configuration field `cancel_probability` stores the class-level eventual cancellation parameter $\bar{\phi}_i$, from which the daily function $\phi_i(\tau)$ is derived in code.

This is exactly the split recommended above. In addition, the simulator now exposes a daily journal with access, booking, cancellation, attendance, and utilization measures, and reports the access metric

$$\mathbb{P}(\tau < T_{\text{access}})$$

for a configurable target such as $T_{\text{access}} = 30$ days.

10 Strategy and Code Review

10.1 Why FCFS is the Right First Strategy

FCFS is the right baseline because it is transparent, easy to explain to operational partners, and reveals the core mechanism of the project with minimal confounding:

$$\text{more demand} \longrightarrow \text{longer offered delay } \tau \longrightarrow \text{more balking / cancellation / no-show.}$$

That makes FCFS an excellent diagnostic benchmark. However, it should not be treated as the final strategic recommendation, because it ignores class heterogeneity. If classes differ materially in access sensitivity or attendance sensitivity, FCFS may be simple but not efficient.

10.2 Why the Code Structure is Sound

The implementation is well aligned with the mathematical formulation for three reasons:

- the policy rule is isolated behind an allocation interface, so FCFS can later be replaced without rewriting the event loop,
- the implementation state is richer than the mathematical state, but only for auditing and measurement; the minimal behavioral content is still the pair (i, τ),
- the simulator exposes arrival logs, cohort logs, slot logs, state logs, and daily journals, so every key equation in the note can be checked against an observable output table.

10.3 Current Strategic and Coding Limits

The main provisional simplification is cancellation. Mathematically, the note now works with the daily function $\phi_i(\tau)$. In code, the current class configuration still starts from a scalar parameter $\bar{\phi}_i$ and derives $\phi_i(\tau)$ internally. That is acceptable for the first prototype because it keeps calibration simple, but if future data support direct estimation of delay-specific cancellation behavior, the natural next step is to let the code accept a cancellation function directly, just as it already does for balking and no-show.

11 References

- Green, L. V., and S. Savin. 2008. Reducing delays for medical appointments: A queueing approach. *Operations Research* 56(6):1526–1538.
- Liu, N. 2015. Optimal choice for appointment scheduling window under patient no-show behavior. *Production and Operations Management* 25(1):128–142.
- Liu, N., and S. Ziya. 2014. Panel size and overbooking decisions for appointment-based services under patient no-shows. *Production and Operations Management* 23(12):2209–2223.